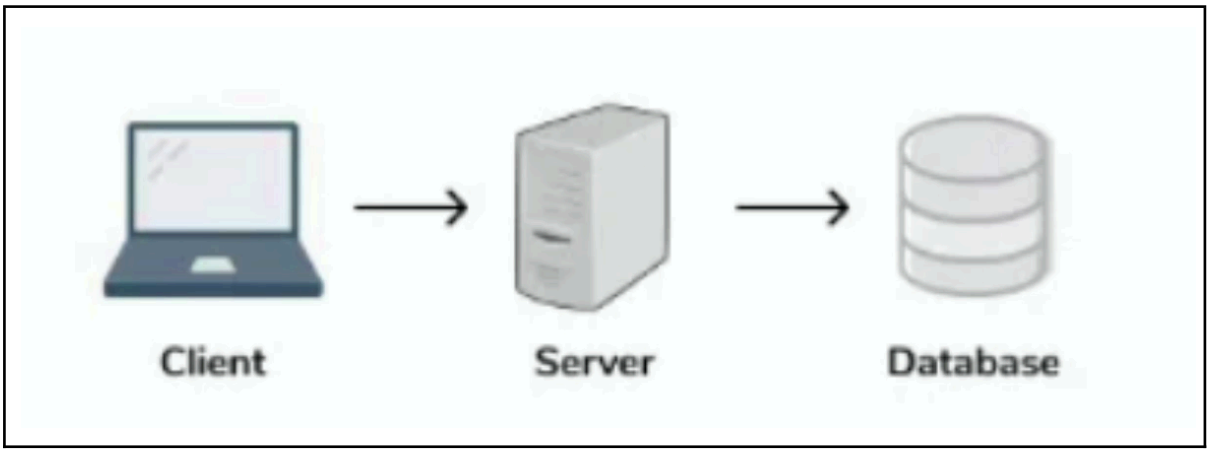
Section 7: Storage in System Design

Understanding Database Models: SQL Vs. NoSQL

In this lecture we will explore the two dominant database paradigms, SQL and NoSQL. Learn how architects choose between them based on consistency, scalability, data structure, and real-world system requirements.

What is a Database?

When we talk about system design, databases are foundational because almost every meaningful application needs data to survive beyond a single request or server restart. A database provides that persistent layer, giving us a reliable way to store information and retrieve it efficiently when the application needs it.

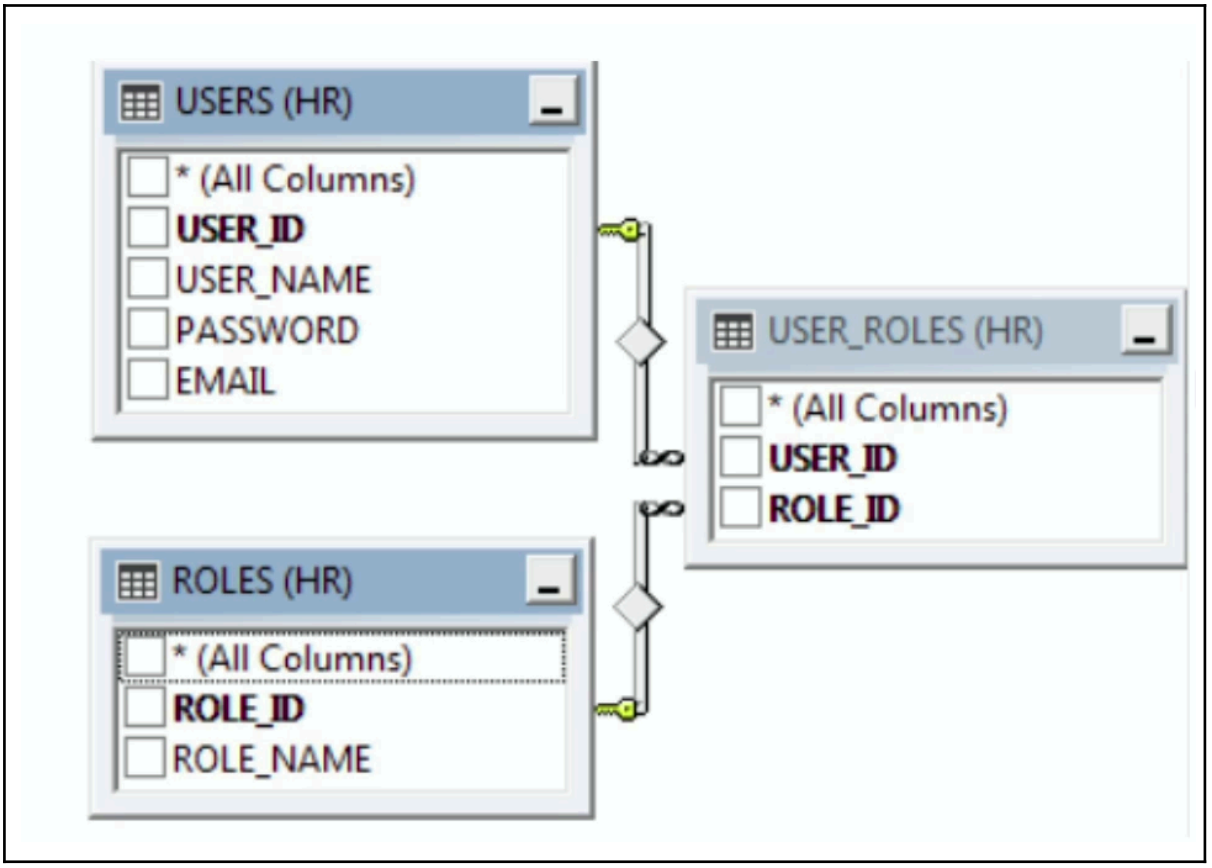


Databases are more than just storage. They allow us to query, filter, update, and manage data in a structured way, which becomes critical as applications grow. So whether you are fetching a user profile, processing an order, or tracking millions of transactions, the database is often at the center of that workflow. From small personal projects to globally distributed platforms, the databases act as the system's source of truth.

As we move forward in this lecture, we will explore how different database technologies make different trade-offs around performance, scalability, consistency and reliability.

Introduction to Relational Databases (SQL)

Relational databases have been the backbone of enterprise applications for decades because they provide highly structured and predictable ways to manage data. Instead of storing information in loosely organized records, they organise data into tables made up of rows and columns, making the relationship between different pieces of data explicit and very easy to manage.

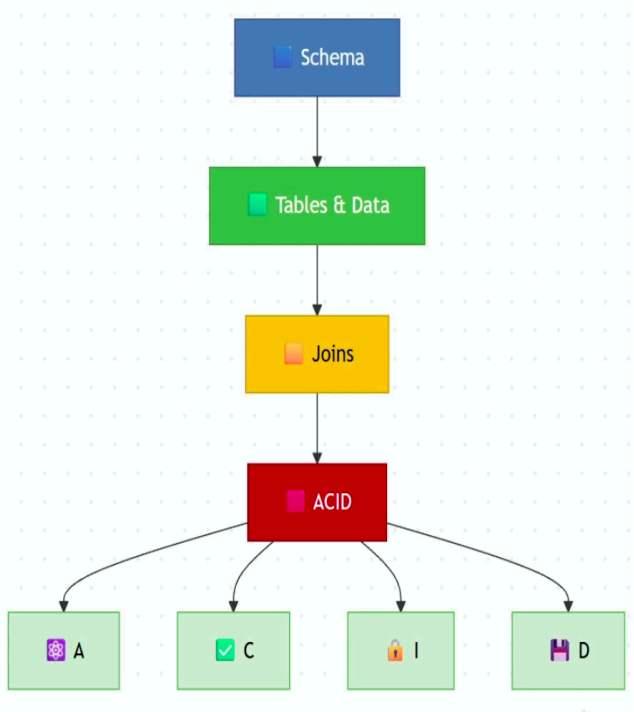


What makes SQL databases particularly powerful is their schema first approach. Before data is stored, we define its structure, what fields exist, their data type, and the rules they must follow. This upfront discipline helps to maintain data quality and consistency as systems grow. The other major advantage is querying power. Using SQL, engineers can filter aggregate, sort, and combine data across multiple tables. With remarkable flexibility.

That is why technologies like MySQL, PostgreSQL, and Oracle remain prominent in domains where data integrity, complex business rules, and transactional accuracy are critical. Domains such as banking, e-commerce, and enterprise systems.

Core Concepts of Relational Databases

The real strength of relational databases comes from 3 foundational ideas - structure, relationship, and transactional reliability.



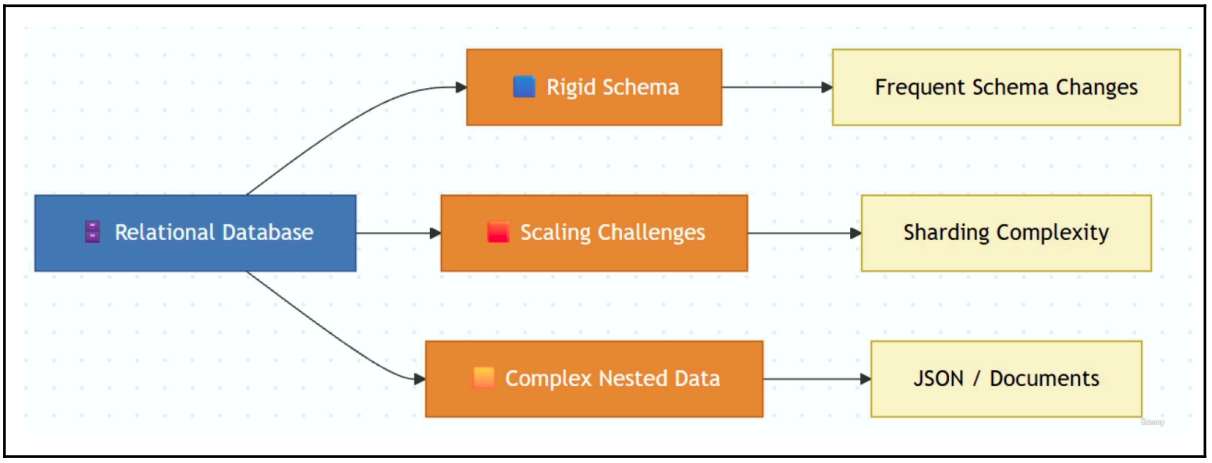
- ➔ **Schemas:** Everything starts with the schema. Before any data is stored, we define the tables, columns, data types, and constraints. This creates a contract for how data should look, helping prevent inconsistencies as applications evolve and multiple teams interact with the same database.

- ➔ **Joins:** In real systems, data is rarely stored in a single table. Users, orders, payments, and products all live separately, and joins allow us to connect those pieces together when answering business questions. This ability to model and query relationships is one of the defining features of relational databases.

- ➔ **ACID transactions:** These guarantees ensure that operations remain reliable even under failures and heavy concurrency. Atomicity ensures that a transaction completes fully or not at all, consistency keeps data valid according to defined rules, isolation prevents concurrent transactions from corrupting each other's work, and durability ensures that committed changes survive crashes and system failures.

Together, schemas, joins and ACID properties are what makes relational databases the preferred choice for systems where correctness and data integrity are non-negotiable, such as banking, payment, health care, and inventory management.

Limitations of Relational Databases



Relational databases solve many important problems, but every architectural choice comes with trade-offs. As systems grow in scale and complexity, some of the strengths of relational databases can also become their limitation. The below are some of the challenges that comes with relational databases:

- **Rapidly evolving model:** As relational databases rely on predefined schemas, adding new fields or changing structures frequently can require migration, coordination, and very careful planning. In environments where data shape changes very often, that rigidity can slow down development.

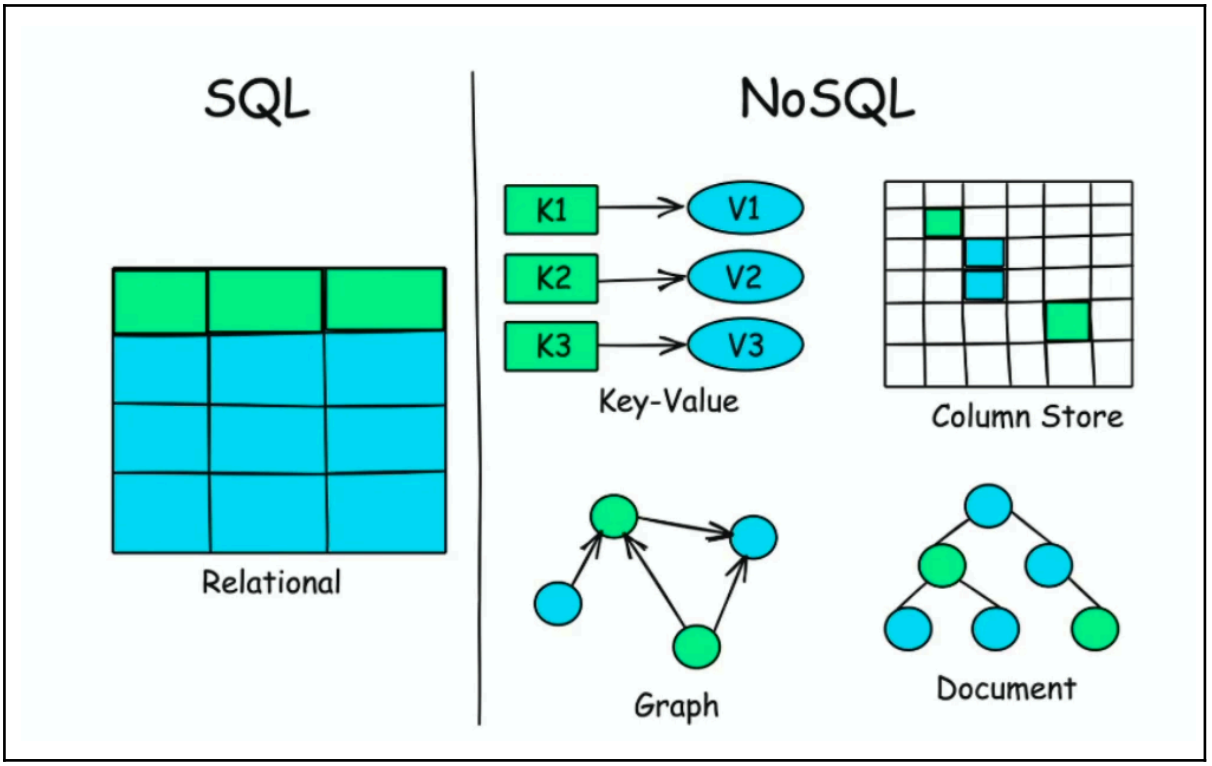
- **Large-scale horizontal scaling:** Relational databases excel on a single powerful server, but distributing data across many machines introduces complexity around sharding, replication, and consistency. As traffic reaches internet scale, scaling a traditional SQL database becomes increasingly difficult and operationally very expensive.

- **Flexible or nested data:** They can also feel less natural when working with highly flexible or deeply nested data. Modern applications often deal with documents, event payloads, user-generated metadata, and JSON structures that don't really neatly fit into rows and columns. While SQL databases have JSON support, relational modeling is not always the most intuitive approach for these workloads.

These limitations don't make relational databases obsolete. They simply highlight that different problems require different storage models. This is exactly why NoSQL databases emerged, offering alternative trade-offs around flexibility, scalability and data modeling.

Introduction to NoSQL

As applications began handling massive amounts of data and serving millions of users, engineers discovered that the traditional relational model was not always best fit. NoSQL databases emerged to address these challenges by prioritizing flexibility, scalability, and performance for specific types of workloads.



The key difference is that many NoSQL databases allow data structures to evolve without requiring a rigid schema definition upfront. This makes them well-suited for rapidly changing applications, user-generated, event data, and other scenarios where the shape of the data is not always predictable.

It is important to understand that NoSQL is not a single technology. It is a family of database models, each optimized for different problems.

- ➔ Document databases store rich JSON-like structures and are ideal for flexible application data.
- ➔ Key-value stores focus on extremely fast lookup and low latency.
- ➔ Column store databases are designed for high throughput and large-scale distributed workloads.
- ➔ Graph databases specialize in traversing complex relationships between entities.

An experienced architect doesn't choose NoSQL because it is newer or scalable. They choose the specific NoSQL model that best matches the application's access pattern, data structure, and scaling requirements. The right database is rarely about popularity. It is about aligning the storage model with the problem that you are trying to solve.

NoSQL - Deep Dive

When people say NoSQ, they are often referring to very different database technologies that solve very different problems. The key is understanding the access pattern each model is optimized for. We have looked at it briefly in the previous section, but let's dive a little deeper.

- **Document databases:** Document databases, such as MongoDB, are designed around application objects. Instead of breaking data into multiple tables, related information can live together inside a single JSON-like document. This makes them particularly effective for user profiles, product catalogs, content management systems, and other domains where data is naturally hierarchical.

- **Key-value databases:** Then key-value databases take simplicity to the extreme. Given a key, a database returns a value with exceptional speed and low latency. Because of this, technologies like Redis, and DynamoDB are commonly used for caching, session management, leaderboards, and high-throughput lookup workloads where response time is critical.

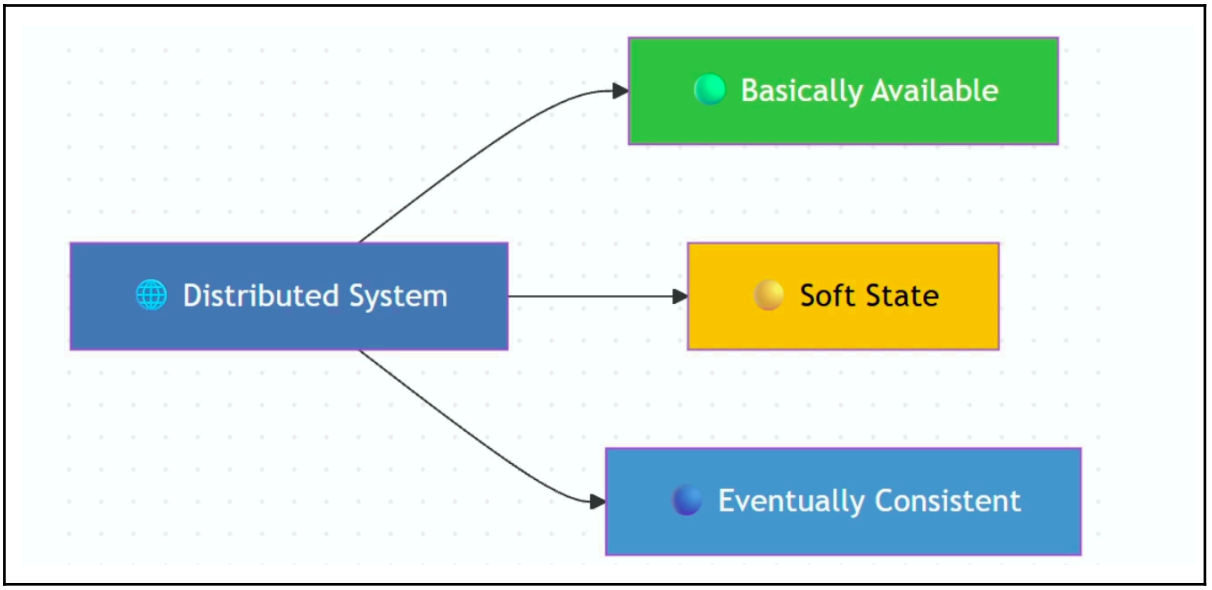
- **Columnar database:** Then columnar databases focus on large-scale distributed data processing. By organizing data differently than traditional row-based systems, they can effectively handle massive write volumes and analyze enormous datasets spread across many servers. This makes them popular for time-series data, event streams, telemetry, and large-scale analytics.

- **Graph databases:** Graph databases approach data from an entirely different perspective. Instead of optimizing for records or documents, they optimize for relationships. When questions involve traversing connections, such as friends of friends, recommendation paths, or fraud detection networks, graph databases like Neo4j can perform operations that would be extremely complex in other databases.

The architectural lesson is that each NoSQL model is optimized around a different access pattern. Before choosing a database, experienced architects first ask, how will the application read and write the data? The answer often determines which database model is the best fit.

BASE Properties of NoSQL

As databases became distributed across multiple servers and even multiple regions, engineers faced a fundamental challenge. Maintaining perfect consistency everywhere can significantly reduce availability and scalability. Many NoSQL systems address this by embracing the BASE model.

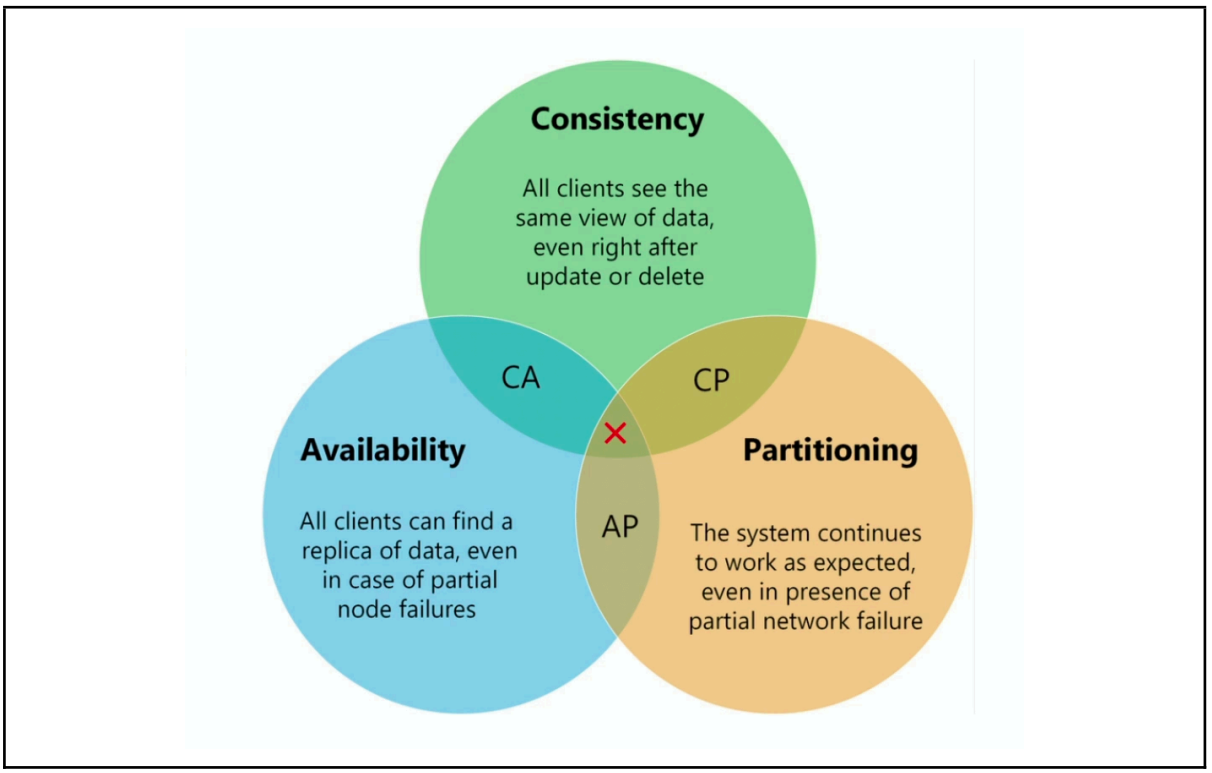


The key idea behind BASE is that, in large-scale distributed systems, being available to users is often more important than having every node reflect the latest update immediately. If a user requests data, the system responds even during failures or network partitions, though that response may not always be the most up-to-date version of data. Because data is replicated across multiple nodes, different replicas can temporarily hold different values. This is what we mean by soft state. The system is continuously converging towards a synchronized view of the data. Over time, updates propagate through the cluster and all replicas eventually agree on the same value, a property known as eventual consistency. This trade-off is common in systems like social media feeds, product catalogs, and caching layers, and large-scale web applications, where serving millions and millions of users reliably is often more important than guaranteeing immediate consistency for every read.

The important architectural lesson is that BASE is not better or worse than ACID. It simply prioritizes a different set of tradeoffs to achieve scalability and higher availability in distributed systems.

CAP Theorem - Revisited

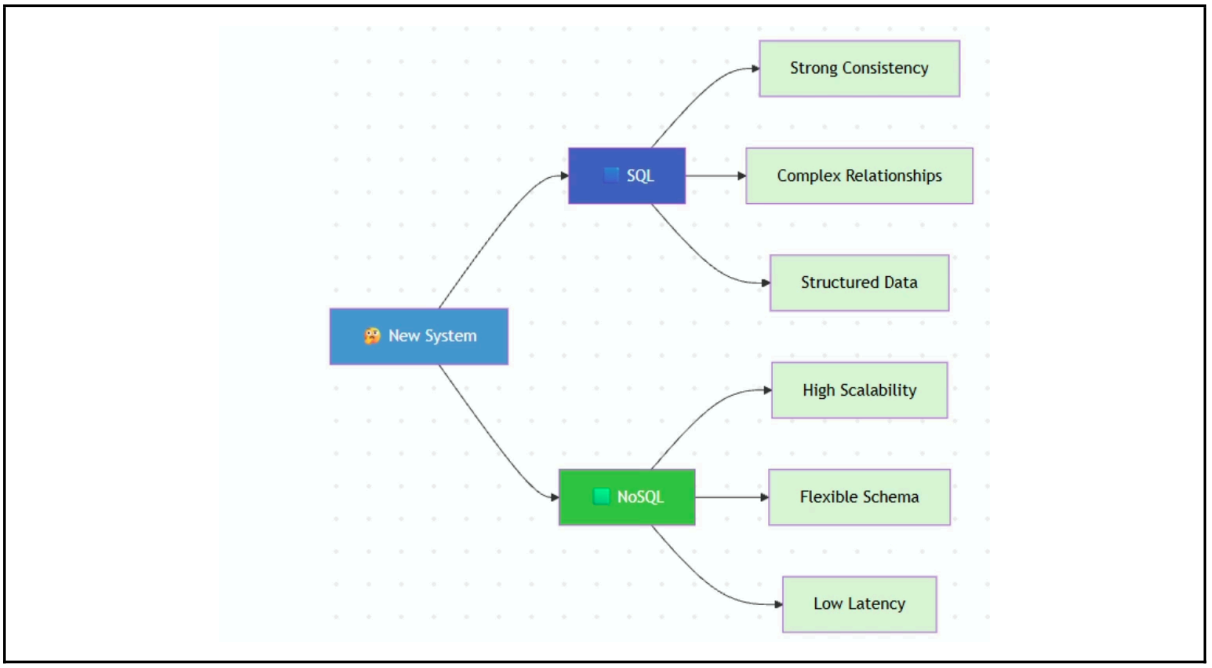
CAP is one of the most important mental models of system design because it explains why distributed databases make different architectural choices. Since data is spread across multiple servers, network failures are no longer hypothetical. They are inevitable. The critical insight is that during a network partition, a system faces a difficult decision. It can prioritize consistency, thereby ensuring that users always see the latest correct data or prioritize availability, ensuring that every request receives a response. Achieving both simultaneously during a partition is impossible. This is why different database technologies make different tradeoffs.



Traditional distributed SQL systems often lead towards consistency, preferring to reject requests rather than risk returning incorrect data. This approach is common in domains like banking, payment, and inventory management, where accuracy is more important that continuous availability. But many no-SQL systems make the opposite choice. They prioritize availability, allowing the system to continue serving requests even if some nodes have not yet received the latest update. The trade-off is temporary inconsistency, which is later resolved through an eventual consistency mechanism.

One important note, CAP is not about choosing CA or AP once and forever. Modern databases often allow tuning these trade-offs based on workloads and business requirements. The real lesson for architects is that every distributed system must decide what matters most during failures, perfect consistency or continuous availability. And that decision should always be driven by business needs and not your technology preference.

When to use what



Choosing between SQL or NoSQL is rarely a question of which database is better. The real question is which database aligns best with the requirements of the system that you are building.

- **When to use SQL?**

➔ SQL databases are typically the right choice when data relationships are important, transactions must be accurate, and consistency cannot be compromised. If your application relies on complex queries, joins, and well-defined rules, as in banking, ERP, inventory, or order management systems, a relational model provides the reliability and structure needed to maintain the data integrity.

- **When to use NoSQL?**

➔ NoSQL databases become attractive when scalability, flexibility, and performance are the primary concern. They excel in environments where data structures evolve frequently, traffic is unpredictable, or the system must handle massive volumes of read and writes. This makes them a natural fit for IoT platforms, recommendation engines, activity feeds, caching layers, and large-scale logging systems.

An important architectural lesson is that modern systems rarely choose one or the other exclusively. A common pattern is polyglot persistence, where different databases are used for different workloads. For example, an e-commerce platform might use a relational database for orders and payments, a document database for product catalog, and Redis for caching. Experienced architects choose the right storage model for each problem rather than forcing every workload into a single database technology.

Summary & Key Takeaways

- SQL and NoSQL are both essential in modern system design
- Choose based on consistency needs, data structure, and scale
- Many systems today use both (polyglot persistence)
- Next:
  - We'll dive deeper into advanced database topics — sharding, replication, and polyglot persistence

Interview Questions

- What are the key differences between SQL and NoSQL databases?
- Explain ACID vs. BASE.
- What are the different types of NoSQL databases, and when would you use each?
- When would you prefer MongoDB over PostgreSQL?
- What are the trade-offs in the CAP theorem?
- Where do SQL and NoSQL databases fit within the CAP theorem categories?
- What database model would you choose for:
  - a. A financial ledger system
  - b. A product catalog
  - c. A real-time chat app
- What are the limitations of relational databases in modern distributed systems?
- What is polyglot persistence and why is it useful?
- How does data modeling differ between SQL and NoSQL systems?